



Assignment 1

Text Processing Fundamentals using MapReduce

Data-Intensive Computing

Tobias Ponesch (11818774)
Jakob Olsacher (01520348)
Alana Hoogmoed (11942398)
Paul Hübner (00371292)
Ekene Edeh (12451120)

May 4, 2026

1 Introduction

Text classification is a foundational task in natural language processing, and the quality of its results depends heavily on how input features - in this case, terms extracted from text - are selected. Not all words carry equal discriminative power across document categories; selecting the right subset is therefore a critical preprocessing step.

This report documents the design, implementation, and evaluation of a MapReduce-based pipeline for term selection from the Amazon Review Dataset 2014¹, developed as part of the Data-intensive Computing course. The dataset comprises 142.8 million product reviews spanning 22 categories, stored in a distributed HDFS environment on a 12-node Hadoop cluster.

The central objective is to compute Pearson's chi-square statistic for all unigram terms relative to each product category, rank terms by their discriminative value, and retain the top 75 per category to ultimately produce a merged term dictionary. The implementation uses the Python-based mrjob framework and is designed to run efficiently at scale.

2 Problem Overview

The task is to identify words that effectively distinguish between different product categories in a large collection of Amazon reviews. Given a corpus of textual reviews, each labeled with a category, the goal is to determine which terms are most strongly associated with each category and therefore most useful for classification.

To achieve this, the data must be processed at scale using the MapReduce paradigm. For each word and category, the chi-square (χ^2) statistic is computed based on their co-occurrence across reviews. In particular, this statistic quantifies how strongly the presence of a word is related to a specific category compared to all other categories.

The final objective is to produce, for each category, a ranked list of the top terms with the highest χ^2 scores. These terms represent the most discriminative features of each category and can serve as a foundation for further text analysis tasks.

3 Methodology and Approach

This section describes the full design and implementation of the project. First, an overview of the pipeline structure is given, followed by a description of the preprocessing steps applied. Then, an explanation of the chi-square metric computation is provided. The section concludes with a detailed account of the MapReduce job design including all key-value pairs.

3.1 Pipeline Overview

The pipeline is structured as three sequential MapReduce jobs, two local processing steps, and a final formatting stage. Figure 1 illustrates the full data flow. Each stage is described below with its input, intermediate, and output key-value pairs.

The pipeline proceeds as follows:

1. Job 1 - distributed counting of all required frequency statistics
2. Side Data Builder - local extraction of global counts from Job 1 output
3. Job 2 - distributed chi-square computation per (category, word) pair
4. Job 3 - distributed selection of top75 terms per category

¹<https://cseweb.ucsd.edu/~jmcauley/datasets/amazon/links.html>

5. Formatter - local production of the final output.txt

3.2 Preprocessing

Preprocessing is applied inside the Job 1 mapper and is therefore fully distributed across the cluster. Each review is processed independently as follows.

- Tokenisation splits the reviewText field into unigrams using the regular expression $[\text{^a-zA-Z<>^}] +$ as a delimiter. This pattern splits on whitespace, digits, and all predefined punctuation characters (including $()\{\}\cdot!?,;:+=-''\#\&*\%\$\$/$), while preserving alphabetic characters and the explicitly retained symbols $<, >, ^$, and $|$.
- Case folding is applied by lowercasing all tokens immediately after splitting, ensuring that, for example, *Battery* and *battery* are treated the same term.
- Stopword filtering removes tokens that appear in the provided stopwords file, which is loaded once per mapper at initialisation and stored as a Python set for $O(1)$ lookup. Additionally, all tokens of length one or fewer are discarded.
- Deduplication within each review is applied by converting the token list to a Python set before counting. This enforces the binary co-occurrence model underlying the chi-square statistic: a term either appears or does not appear in a review, and multiple occurrences within the same review are not counted.

3.3 Chi-Square Metric Computation

The chi-square statistic $\chi^2(w, c)$ measures the degree of statistical dependence between a term w and a category c . For each (term, category) pair, a 2×2 contingency table is constructed.

	In category c	Not in category c
Contains w	A	B
Does not contain w	C	D

Table 1: Contingency Table

Where:

- A = number of reviews in category c that contain term w
- B = number of reviews not in category c that contain term w
- C = number of reviews in category c that do not contain term w
- D = number of reviews neither in category c nor containing term w
- N = total number of reviews (sum of A, B, C, D)

The chi-square value is then computed as follows:

$$\chi^2(w, c) = \frac{N \cdot (AD - BC)^2}{(A + B)(A + C)(B + D)(C + D)}$$

Cases where the denominator equals zero are explicitly skipped to avoid division-by-zero errors. This occurs when a term appears in every review or in no review, rendering the statistic undefined.

The computation requires three global quantities that cannot be derived from a single review in isolation: the total number of reviews N , the per-category review count cat_{total} , and the per-word document frequency w_{total} . The pipeline therefore separates the counting phase (Job 1) from the computation phase (Job2).

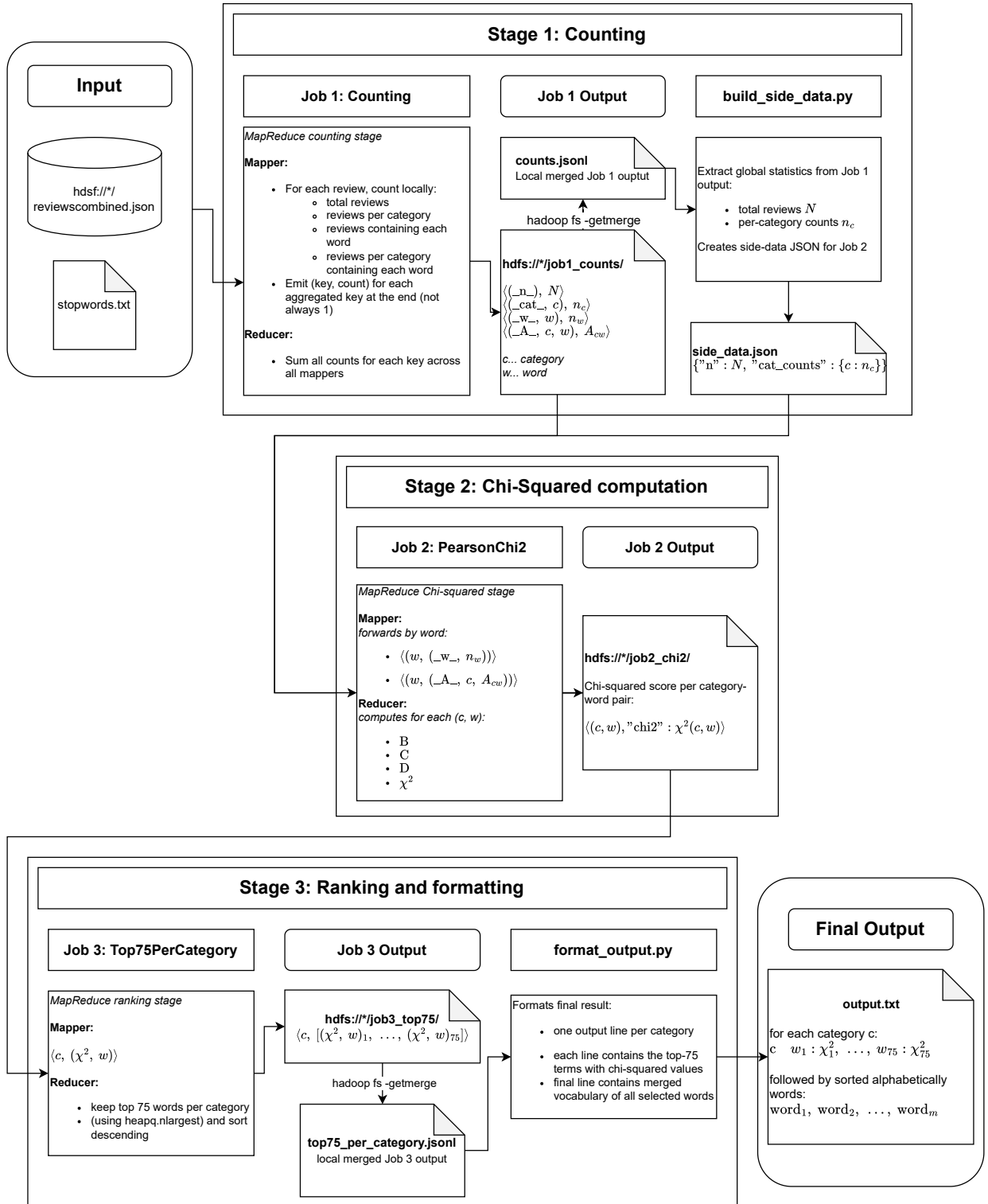


Figure 1: MapReduce pipeline for chi-squared computation

3.4 MapReduce Design

3.4.1 Job 1: Frequency Counting

Job 1 reads the raw review JSON file from HDFS and computes all frequency counts required for the chi-square formula: the total number of reviews N , the per-category review count, the per-word document frequency, and the per-(category, word) co-occurrence count.

Mapper. For each input review, the mapper emits four key types after applying the full preprocessing pipeline described in Section 3.2:

Key	Value	Meaning
["_n_"]	1	One review seen
["_cat_", c]	1	One review seen in category c
["_w_", w]	1	Term w seen in one review
["_A_", c, w]	1	Term w seen in one review in category c

Table 2: Job 1: Mapper Output

To reduce the volume of data passed to the shuffle and sort phase, the implementation applies the in-mapper combining optimization pattern. Each mapper instance maintains a local count dictionary that aggregates term frequencies across reviews in memory, flushing to the output stream either upon reaching a buffer threshold of 100,000 entries (to guard against memory exhaustion) or upon completion of the mapper’s input split.

Combiner and Reducer. Both the combiner and reducer apply the same logic: sum all values for each key. The reducer therefore produces globally aggregated counts for all four key types. Job 1 is configured with 24 reducers to parallelise this aggregation across the cluster.

Output: one HDFS record per key, containing the global count for that key.

3.4.2 Side Data Builder

After Job 1 completes, its output is fetched from HDFS to the local filesystem. A local Python script then makes a single pass through this output, extracting only the `_n_` and `_cat_` records and writing them to a compact JSON file. This file contains the total review count N and the per-category review counts, which are the global statistics required for the subsequent reducers.

This design decision avoids the need for an additional MapReduce job solely to broadcast global counts. Since there are only 22 categories, the output of this step is small enough to be distributed to all reducer nodes via mrjob’s file distribution mechanism and loaded into memory at reducer initialisation.

3.4.3 Job 2: Chi-Square Computation

Job 2 reads the Job 1 HDFS output and computes $\chi^2(w, c)$ for every valid (word, category) pair.

Mapper. Re-keys Job 1 output records by word alone, emitting two value types per word:

Key	Value	Meaning
w	["_w_", count]	Global document frequency w
w	["_A_", c , count]	Co-occurrence count of w in category c

Table 3: Job 2: Mapper Output

Grouping by word ensures that each reducer receives both the global word frequency and all per-category co-occurrence counts for a given term in a single reduce call, making

it possible to compute all (w, c) chi-square values for word w without any additional data access.

Reducer. Loads the output from the side data builder at initialisation to obtain N and all category counts. For each word, it iterates over all associated category entries, derives B, C and D from the available counts, and computes and emits $\chi^2(w, c)$:

Key	Value
$[c, w]$	$\{ \text{"chi2"}: \chi^2 \text{ value} \}$

Table 4: Job 2: Reducer Output

3.4.4 Job 3: Top 75 Selection

Job 3 reads Job 2’s output and selects the 75 highest χ^2 terms per category.

Mapper. Re-keys each record by category alone, emitting $(\text{chi2}, \text{word})$ value pairs. This routes all chi-square scores for a given category to the same reducer.

Reducer. Applies Python’s `heapq.nlargest(75, values)` to select the top 75 entries without sorting the full value list, operating in $O(n \log k)$ time where $k = 75$. The selected entries are then sorted in descending order of χ^2 for output.

Key	Value
c	$[(\chi_1^2, w_1), \dots, (\chi_{75}^2, w_{75})]$

Table 5: Job 3: Reducer Output

3.4.5 Formatter

A final local script reads Job 3’s merged output and produces `output.txt` in the required format: one line per category in alphabetical order, with terms listed as `word: chi2.value` in descending order, followed by a single line containing all unique terms across all categories sorted alphabetically. A tie-breaking step ensures that terms with identical χ^2 values within a category are listed in alphabetical order.

4 Results

The pipeline was executed on the development dataset and produces a correctly formatted output file containing one line per category and a final merged dictionary line. The output was verified against a reference implementation, which confirmed that all chi-square values match within a tolerance of $\delta = 10^{-9}$.

The output covers all 22 categories present in the dataset. Table 6 shows the top three terms and their corresponding χ^2 values for a representative selection of categories:

The results are broadly consistent with intuitive expectations: terms with the highest χ^2 values are very domain-specific (e.g., `diaper`, `music`), while lower-ranked terms within each category tend to be more general (e.g., `favorite`). The merged dictionary on the final line of the output file contains all unique terms appearing in any of the 75 term category lists, sorted alphabetically.

Runtime Performance. The complete MapReduce pipeline was executed on the Hadoop cluster using the full dataset. The total processing time, measured from the start of the first job to the completion of the final output, was: **730 seconds (12 minutes and 10 seconds)**.

Category c	Term w	χ^2
Automotive	oem	1,068.86
	honda	1,035.22
	engine	763.28
Baby	diaper	2,429.74
	crib	2,411.47
	newborn	1,292.01
CDs & Vinyl	music	13,083.77
	albums	6,705.82
	tracks	6,516.95

Table 6: Top 3 χ^2 terms for a selection of categories on the development dataset

5 Discussion

The output produced on the development set matches the reference implementation precisely, confirming that the chi-square computation, preprocessing pipeline, and top-k selection are correctly implemented. The tie-breaking logic in the formatter, which enforces alphabetical ordering among terms sharing identical chi-square values, ensures deterministic output regardless of the order in which records arrive from HDFS.

5.1 Design Choices and Efficiency Considerations

Several explicit design decisions were made to maximise runtime efficiency on the full dataset:

In-mapper combining in Job 1 reduces intermediate output volume significantly compared to emitting one record per (category, word) pair per review. Since the Amazon dataset contains reviews with many overlapping terms across consecutive input records, local aggregation before the shuffle phase substantially reduces network I/O.

Side data distribution replaces what would otherwise be a fourth MapReduce job for broadcasting global counts. The output file from this stage is 22 entries in size and fits trivially in reducer memory, making it far more efficient to distribute as a file than to join through HDFS.

Word-keyed grouping in Job 2 ensures that each reducer can compile chi-square values for all categories associated with a given word in a single pass, avoiding redundant data reads or multiple jobs over the same input.

heapq.nlargest in Job 3 avoids a full sort of potentially millions of (category, word) entries per reducer. For $k = 75$ out of potentially hundreds of thousands of terms, this is substantially faster than sorting the entire list.

Last but not least, Job 1 parallelism is explicitly set to 24 reducers, distributing the aggregation workload across the cluster and preventing a single reducer from becoming a bottleneck on high-frequency keys.

6 Limitations

The approach relies on a unigram (bag-of-words) model, which ignores context and word order, limiting semantic understanding. Using document frequency instead of term frequency may overlook word importance within documents, while simple regex-based tokenization and dependence on a fixed stopwords list can reduce text quality. Additionally, the chi-square statistic may overemphasize rare terms, and the in-mapper combiner introduces memory constraints. Finally, the use of a single reducer in the final stage and the absence

of advanced features such as n-grams or semantic representations may limit scalability and overall performance.

6.1 Practical Implications

The resulting term lists serve as a feature vocabulary for downstream text classification tasks, where selecting informative features is critical for model performance. The chi-square statistic provides a computationally tractable and theoretically grounded criterion for this selection at scale. The top-ranked terms per category are strongly domain-specific and would function as effective discriminative features in a classifier trained on this data.

The pipeline architecture is not specific to the Amazon dataset. The three-job structure (counting, computing, selecting) generalises to any corpus where chi-square feature selection over categorical labels is required. With minor modifications to the input parsing step, the same pipeline could be applied to other large-scale text corpora such as scientific literature or social media data, provided the data can be represented as labelled documents in a line-delimited format.

6.2 Future Work

Several enhancements could improve the current implementation in future work. Extending the tokenisation to include n-grams or phrase-based features would capture multi-word expressions that unigrams miss, at the cost of a significantly larger feature space and higher computational load. Complementing chi-square with TF-IDF weighting would introduce within-document term frequency into the scoring, potentially surfacing rare but highly category-specific terms that the binary co-occurrence model misses. More advanced linguistic preprocessing could reduce vocabulary size and collapse morphological variants into single features. Additionally, filtering terms below a minimum document frequency threshold before chi-square computation would reduce noise from very rare terms and decrease the volume of data processed in Jobs 2 and 3. Finally, extending the pipeline to support streaming or real-time processing would allow the feature vocabulary to be updated incrementally as new data arrives, making the system applicable to production environments.

7 Conclusion

This project presents a scalable MapReduce pipeline for chi-square-based term selection over the Amazon Review Dataset from 2014. The pipeline computes discriminative scores for all (term, category) pairs across 22 product categories, selects the top 75 terms per category, and produces a merged dictionary. This has been evaluated by value comparison with the reference implementation on the development set. Key design decisions, including in-mapper combining, local side data extraction, word-keyed reducer grouping, and heap-based top-k selection, collectively minimize shuffle volume and reduce the number of distributed jobs required.

The implementation demonstrates that careful key-value design and targeted optimization at each stage are sufficient to process text at scale within practical time constraints. While the chi-square statistic has known limitations (in particular its bias toward high frequency terms and its binary co-occurrence model) these are properties of the metric itself rather than the implementation. Future work could explore alternative weighting schemes such as TF-IDF or extend the pipeline to support n-gram features and more linguistically sophisticated pre-processing.